

SOLUTIONS – OOADI

LECTURE 9

-
1. The current implementation of the train program has CommandLineInputHandler and TrainListManager talk directly through a shared object reference. To split them across two programs we need to replace that direct reference with some form of network communication.

BEFORE:

CommandLineInputHandler → direct method call → TrainListManager

AFTER:

Client program

Server program

CommandLineInputHandler → network (socket) → TrainListManager

If we add only a simple socket approach we can get by with something like this on the server side:

```
import java.io.*;
import java.net.*;

try (ServerSocket serverSocket = new ServerSocket(PORT)) {
    Socket clientSocket = serverSocket.accept(); // accept() blocks until a client connects
    System.out.println("Client connected: " + clientSocket.getInetAddress());
} catch { ... }
```

And similarly on the client side:

```
import java.io.*;
import java.net.*;

try (Socket socket = new Socket(HOST, PORT);
    BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
    PrintWriter out = new PrintWriter(
        new OutputStreamWriter(socket.getOutputStream()),
        true); // auto-flush
    BufferedReader userInput = new BufferedReader(
        new InputStreamReader(System.in))) { ... } catch { ... }
```

Here we have no less than four resources in one try() block.

1. With new Socket() we reach out to the server to establish connection
 2. With new BufferedReader() we read from the server
 3. With new PrintWriter() we write to the server
 4. With new BufferedReader() we start to read from the user
-

The try-with-resources means that Java will handle all the closing down of the resources for us. And we need to do the closing in the right order. If we close the socket before flushing the PrintWriter, the last message will get lost. So, the order matters. Adding all those resources mean that we engage several wrappers to bridge bytes and text, to make the code cleaner. You can find my suggested solution on Moodle. Run the server from the IDE and the client from a terminal using

```
jhm$ javac lectuer9Client/TrainClient.java
jhm$ java lectuer9Client.TrainClient
```

-
2. Considering distribution of responsibility then the task of managing a file of all trains should belong to the TrainListManager alone. This makes it easy to add the functionality. I have chosen to save every time the list changes and then using a simple text format.

Server side:	jhm\$ java lectuer9Client.TrainClient
No saved fleet found - loading sample fleet	Train Client connecting to localhost:5001
Fleet saved to fleet.txt (5 trains)	Connected! Commands: list / search / add / delete / quit
Train Server starting on port 5001	Command: add
Waiting for client...	Train ID: D-666
Client connected: /127.0.0.1	Type (DIESEL/ELECTRIC): DIESEL
Received command: list	Number of wagons: 1
Received command: add D-666 DIESEL 1 666 666	Wagon 1 length (m): 666
Added: DieselTrain{id=D-666, type=DIESEL, len=686,0m, wt=746,0t}	Wagon 1 weight (t): 666
Fleet saved to fleet.txt (6 trains)	Added: DieselTrain{id=D-666, type=DIESEL, len=686,0m, wt=746,0t}
Received command: quit	Command: quit
Client requested quit. Shutting down.	BYE
	Disconnected from server.

This then produces a fleet.txt file like the following

```
DIESEL,D-101,14.0:15.0|14.0:15.0|14.0:15.0|14.0:15.0
ELECTRIC,E-201,75,26.0:40.0|26.0:40.0|26.0:40.0
DIESEL,D-102,24.0:35.0|16.0:16.0
ELECTRIC,E-202,15,14.0:15.0|14.0:15.0|14.0:15.0|14.0:15.0
ELECTRIC,E-203,55,24.0:35.0|16.0:16.0
DIESEL,D-666,666.0:666.0
```

-
3. a) To allow for “real” networked operation we basically just need to change away from the localhost approach. Remember that a real network introduces latency, the need to negotiate with firewall settings and IP discovery. If you just manually find the IP of your server machine and then prompts the client with this, then you should be good to go .. but remember to handle any firewall issues. On Moodle I have the TrainClientNetworked.java as an example.
-

- b) With the example code on Moodle, the TrainClientGUI.java file, you should see some thing like this, depending on how you chose to connect the server and client.

Speed	Behaviour
Normal clicks	Works perfectly
Moderate rapid clicks	Rejected clicks counter increments, button flashes red
Very fast clicks	Many rejections, only one request in flight at a time

- c) Using something like my TrainClienAuto.java you should see things fork okay down to approximately 10ms delay. Lower than that you will start to see that the server cannot keep up. As for the TCP versus UDP thing, then UDP would break this program immediately. The END marker the client waits for in readLine() could simply never arrive — the packet gets dropped. The client blocks forever. TCP's guaranteed delivery is not optional for this protocol — it's the foundation everything else depends on.

